Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información
Maestría en Seguridad Informática

TAREA #6 · Actividad S06

Auditoría y Endurecimiento de APIs y Microservicios

Estudiante	Emilio Andrés Hernández
Carné	1593-20-19375
Correo	ehernandez10@miumg.edu.gt
Catedrático	Luis Enrique Rivas Ventura
Curso	Seguridad en Aplicaciones
Fecha	4 de noviembre de 2026

Entregable (ZIP Actividad_S06_Hernandez_Emilio):
https://drive.google.com/file/d/10N_6WDHSJb7Q3-Hgwk-hVhsz1N3-0xhR/view?usp=sharing
Acceso: cualquier persona de la Universidad Mariano Gálvez con el enlace.

Repositorio: https://github.com/emiliohernandezdev/t6-lab-seg-aplicaciones

Contenido

1. Acceso al entregable
2. README – laboratorio, ejecución y mapa de evidencias
3. Inventario de endpoints de crAPI
4. Hallazgos
5. Relación con el caso Optus
6. Resultado de pruebas automatizadas (pytest)

1. Acceso al entregable

El código, las configuraciones y las evidencias completas se entregan en un único ZIP con la estructura solicitada (`Actividad_S06_Hernandez_Emilio/`: README.md, docker-compose.yml, src/, gateway/, tests/, scripts/, evidencias/, hallazgos.md, caso_optus.md). Las claves privadas no se incluyen; se regeneran con `scripts/gen_certs.sh`.

Google Drive: https://drive.google.com/file/d/10N_6WDHSJb7Q3-Hgwk-hVhsz1N3-0xhR/view?usp=sharing

GitHub: https://github.com/emiliohernandezdev/t6-lab-seg-aplicaciones

TAREA #6 – Auditoría y Endurecimiento de APIs y microservicios

Laboratorio local con dos partes:

- **Bloque A:** auditoría de OWASP crAPI. Inventario y hallazgos en `hallazgos.md`, evidencias en `evidencias/crAPI/`. Para reproducirlo: clonar crAPI en `C:\labs\crAPI`, `docker compose -f docker-compose.yml --compatibility up -d` y luego `bash scripts/evidencias_crapi.sh` (registra los dos usuarios, da de alta sus vehículos leyendo el pincode de MailHog y captura las pruebas). La captura del navegador está en `evidencias/crAPI/crapi_funcionando.png`.
- **Bloque B:** servicio de pedidos propio (`orders-api`), con Keycloak, un gateway Nginx, mTLS y GraphQL. Es lo que levanta este `docker-compose.yml`.

```
cliente ──HTTPS──> gateway (nginx, 127.0.0.1:8443) ──mTLS──> orders-api (FastAPI + Strawberry)
                                                                  │
                          keycloak (127.0.0.1:8080) <── JWKS ─────┘
```

`orders-api` no publica ningún puerto. Vive en la red `backend` (internal) y solo acepta conexiones con un certificado de cliente firmado por la CA del laboratorio.

Requisitos

- Docker Desktop (con Compose v2)
- Git Bash, para correr los scripts `.sh` (trae `openssl`)
- No hace falta tener Python instalado: las pruebas corren en un contenedor.

Cómo levantarlo

```
bash scripts/gen_certs.sh          # 1. CA y certificados (solo la primera vez)
docker compose up -d --build       # 2. keycloak + orders-api + gateway
```

Keycloak tarda unos 30 s en arrancar. `orders-api` espera a que esté listo.

Pruebas

```
docker compose --profile test run --rm tests
```

Desde **Git Bash**, si se le pasan rutas al contenedor, hay que anteponer `MSYS_NO_PATHCONV=1`. Desde PowerShell funciona tal cual.

Son 75 pruebas y tardan alrededor de 1 minuto, porque algunas esperan a que venzan la ventana del rate limit o un token.

Para regenerar todas las evidencias (antes con `INSECURE_MODE=true`, después, pytest y comprobaciones desde el host):

```
bash scripts/evidencias.sh
```

Obtener un token a mano

```
bash scripts/get_token.sh usuarioa            # imprime el access token
TOKEN=$(bash scripts/get_token.sh usuarioa)
curl --cacert certs/ca.crt -H "Authorization: Bearer $TOKEN" https://localhost:8443/api/orders/1
```

En Windows, si `curl` da error de revocación con la CA local, agregar `--ssl-no-revoke`.

Usuarios de prueba del realm `lab`: `usuarioa` (user), `usuariob` (user) y `admin` (user, admin). Credenciales ficticias definidas en `keycloak/realm-lab.json`.

Hay además un realm `otro` (usuario `intruso`), que solo sirve para la prueba de emisor incorrecto, y un cliente `lab-cli-corto` que emite tokens de 5 s para la prueba de token vencido.

Endpoints publicados por el gateway

Método	Ruta	Control
GET	`/api/orders`	scope `orders:read`, solo pedidos propios
POST	`/api/orders`	scope `orders:write`, esquema estricto
GET	`/api/orders/{id}`	scope `orders:read`, dueño = `sub` (BOLA), 10 req/10 s por usuario
GET, PATCH	`/api/profile`	lista blanca de campos de entrada y de salida
GET	`/api/admin/audit`	rol `admin` (BFLA)
POST	`/graphql`	profundidad 3, `first` ≤ 20, costo ≤ 500, sin introspección en prod

Cualquier otra ruta, incluidas `/healthz` y `/docs` de la propia API, devuelve 404 en el gateway.

Antes / después

`INSECURE_MODE=true` apaga los controles del servicio: no valida la firma del JWT, no revisa dueño ni rol, devuelve el perfil completo, acepta cualquier campo y no aplica ni rate limit ni límites de GraphQL. Se usa solo para la evidencia de "antes":

```
INSECURE_MODE=true docker compose up -d --force-recreate orders-api
```

Mapa de evidencias

Punto	Carpeta	Pruebas
1–3 crAPI	`evidencias/crAPI/`	`scripts/evidencias_crapi.sh` (inventario, BOLA, exposición de datos)
5 BOLA/BFLA/propiedades	`evidencias/BOLA_BFLA/`	`tests/test_bola_bfla.py`
6 OIDC/JWT	`evidencias/JWT_OIDC/`	`tests/test_jwt.py`
7 esquema y rate limit	`evidencias/esquema_rate_limit/`	`tests/test_schema.py`, `tests/test_rate_limit.py`

Punto	Carpeta	Pruebas
8–9 gateway y mTLS	`evidencias/gateway_mTLS/`	`tests/test_mtls.py`
10 GraphQL	`evidencias/graphql/`	`tests/test_graphql.py`
11 ejecución completa	`evidencias/pytest_resultado.txt`	todas

Cada carpeta tiene `antes.txt` / `despues.txt` (solicitud y respuesta en formato legible) y su versión `.json` con los mismos datos. Los tokens aparecen recortados (`eyJ…[OCULTO]`).

Notas

- Las claves privadas (`certs/*.key`) no se entregan. Se regeneran con `scripts/gen_certs.sh`.
- Los datos viven en memoria: al recrear `orders-api` vuelven a su estado inicial.
- Para apagar todo: `docker compose down -v`.

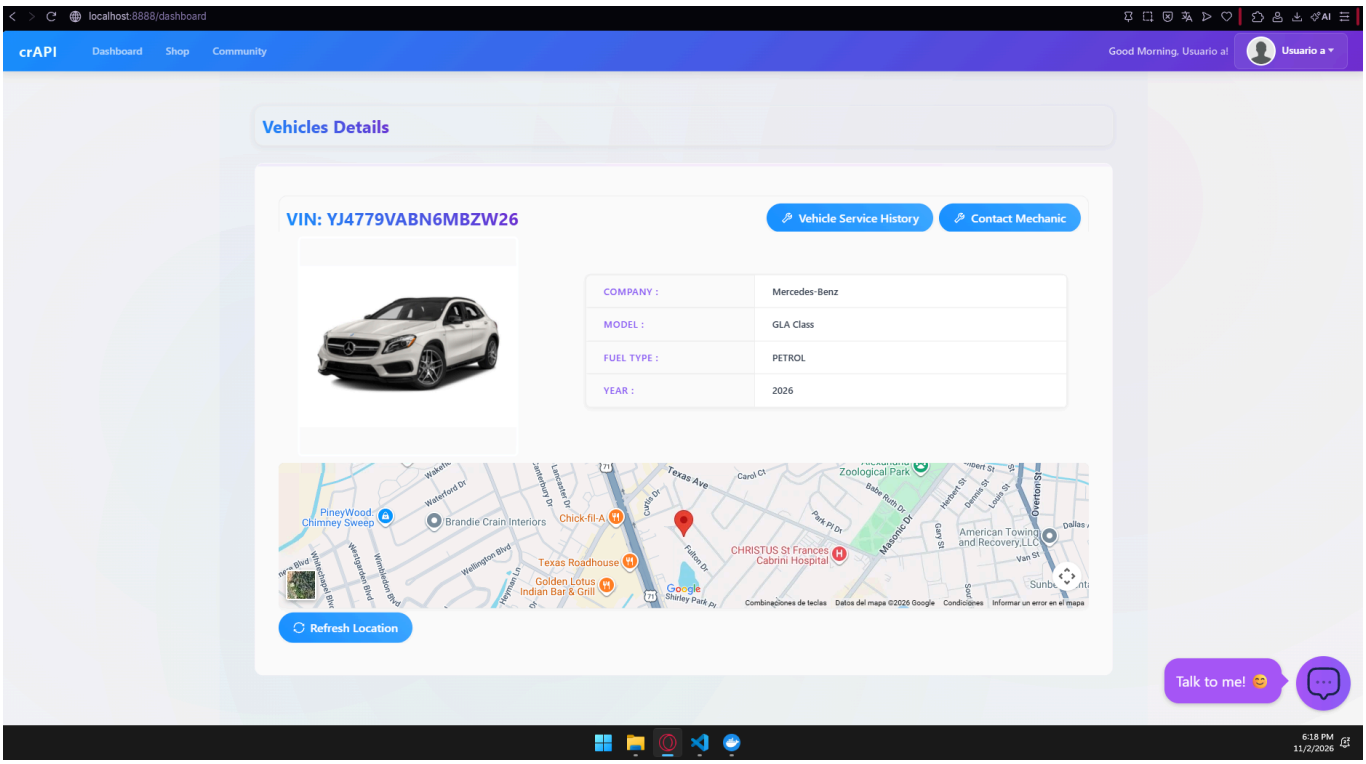
Inventario de endpoints – crAPI

Observado en mi propio tráfico (curl) iniciando sesión con Usuario A y Usuario B. - crAPI: commit `b5fc307` (2026-09-09), levantado en `C:\labs\crAPI\deploy\docker` - Base: `http://localhost:8888` - Fecha de captura: 2026-11-03

#	Método	Ruta	Parámetros relevantes	Autenticación	Qué devuelve
1	POST	`/identity/api/auth/signup`	name, email, number, password	ninguna	confirmación de registro
2	POST	`/identity/api/auth/login`	email, password	ninguna	JWT (`token`)
3	GET	`/identity/api/v2/user/dashboard`	–	Bearer JWT	id, name, email, number, credit, role
4	POST	`/identity/api/v2/vehicle/add_vehicle`	vin, pincode	Bearer JWT	registra el vehículo del usuario
5	GET	`/identity/api/v2/vehicle/vehicles`	–	Bearer JWT	vehículos del usuario (incluye `uuid`)
6	GET	`/identity/api/v2/vehicle/{uuid}/location`	uuid (path)	Bearer JWT	ubicación GPS, nombre y correo (**H1 – BOLA**)
7	GET	`/community/api/v2/community/posts/recent`	limit, offset	Bearer JWT	posts con `email` y `vehicleid` del autor (**H2**)
8	GET	`/workshop/api/shop/products`	–	Bearer JWT	catálogo y crédito
9	GET	`/workshop/api/mechanic/`	–	Bearer JWT	lista de mecánicos (código y correo)
10	GET	`/workshop/api/mechanic/mechanic_report`	report_id (query, secuencial)	Bearer JWT	reporte con VIN, correo y teléfono del dueño (**H1b – BOLA**)
11	POST	`/community/api/v2/community/posts`	title, content	Bearer JWT	post creado
12	GET	`/workshop/api/shop/orders`	limit, offset	Bearer JWT	pedidos del usuario

crAPI funcionando (punto 1)

Dashboard de crAPI con la sesión de Usuario A (`localhost:8888/dashboard`), mostrando su vehículo (VIN `YJ4779VABN6MBZW26`) y la ubicación en el mapa:



Archivos de evidencia (tokens recortados a `eyJ…[OCULTO]`)

- `entorno.txt` — `docker --version`, SO y `docker compose ps` de crAPI (punto 1)
- `crapi_funcionando.png` — captura de crAPI en el navegador (`localhost:8888/dashboard`), sesión de Usuario A, con el vehículo VIN `YJ4779VABN6MBZW26` y su ubicación en el mapa
- `recon_dashboard_A.txt`, `recon_shop_products.txt`, `recon_mechanics.txt` — reconocimiento
- `H2_community_posts_recent.txt` — exposición de datos
- `H1_location_propio_A.txt` + `H1_location_vehiculo_B.txt` — par BOLA (objeto propio vs. ajeno)
- `H1b_mechanic_report_1.txt` — BOLA por id secuencial
- `neg_location_sin_token.txt` — control negativo (401)

Todo esto se regenera con `bash scripts/evidencias_crapi.sh` (con crAPI arriba y los dos usuarios creados).

Hallazgos

Inventario de endpoints de crAPI: `evidencias/crAPI/inventario_endpoints.md`

Bloque A – crAPI

Reproducido con dos usuarios de prueba (`usuarioa@lab.local`, `usuariob@lab.local`), ambos con un vehículo registrado con el VIN y pincode que crAPI envía por correo (MailHog). Entorno y `docker compose ps` en `evidencias/crAPI/entorno.txt`; captura de crAPI funcionando (sesión de A, con su vehículo y ubicación) en `evidencias/crAPI/crapi_funcionando.png`:

H1 – BOLA en la ubicación del vehículo

- **Endpoint:** `GET /identity/api/v2/vehicle/{vehicleId}/location`
- **Condición:** el servidor no comprueba que el vehículo pertenezca al usuario del token; basta cambiar el `uuid` del vehículo.
- **Impacto:** con la sesión de A se obtiene la ubicación GPS, el nombre y el correo del vehículo de B (HTTP 200).
- **Evidencia:** `evidencias/crAPI/H1_location_propio_A.txt` (A sobre su vehículo, baseline) y `H1_location_vehiculo_B.txt` (A sobre el vehículo de B). Control negativo sin token: `neg_location_sin_token.txt` (401).
- **Corrección:** validar el dueño del objeto contra el `sub` del token. Aplicado en mi servicio en `GET /api/orders/{id}` (ver H4).

H1b – BOLA en el reporte del mecánico (IDs secuenciales)

- **Endpoint:** `GET /workshop/api/mechanic/mechanic_report?report_id=N`
- **Condición:** el `report_id` es secuencial y no se valida la propiedad; A lee el reporte `1`.
- **Impacto:** se expone el VIN, el correo y el teléfono del dueño del reporte (en la semilla, los del usuario `admin`).
- **Evidencia:** `evidencias/crAPI/H1b_mechanic_report_1.txt` (HTTP 200).
- **Corrección:** misma que H1 (dueño del objeto = `sub`) y además evitar identificadores secuenciales predecibles.

H2 – Exposición excesiva de datos en el feed de la comunidad

- **Endpoint:** `GET /community/api/v2/community/posts/recent`
- **Condición:** cada post incluye datos del autor que la pantalla no muestra: `email` y `vehicleid`.
- **Impacto:** cualquier usuario autenticado recolecta correos e IDs de vehículos ajenos, que es justo lo que habilita H1.
- **Evidencia:** `evidencias/crAPI/H2_community_posts_recent.txt`
- **Corrección:** modelo de respuesta con lista blanca. En mi servicio, `ProfileOut` (ver H6).

> La **asignación masiva** (propiedades que solo debería controlar el servidor) se demuestra de forma controlada en mi propio servicio: ver H7 (`PATCH /api/profile` con `role=admin`), con

> evidencia antes/después y prueba automatizada.

Bloque B – mi servicio (antes de corregir, `INSECURE_MODE=true`)

H4 – BOLA en pedidos

- **Endpoint:** `GET /api/orders/{id}`
- **Condición:** solo se buscaba el pedido por id, sin comparar el dueño.
- **Impacto:** usuarioa lee el pedido 3 de usuariob, con dirección y artículos (200).
- **Evidencia:** `evidencias/BOLA_BFLA/antes.txt` → `despues.txt`
- **Corrección:** `owner_id == sub` y 404 en caso contrario. Prueba: `test_bola_bfla.py`.

H5 – BFLA en auditoría

- **Endpoint:** `GET /api/admin/audit`
- **Condición:** la ruta solo exigía estar autenticado.
- **Impacto:** un usuario común lee la auditoría (200).
- **Evidencia:** `evidencias/BOLA_BFLA/antes.txt` → `despues.txt` (403)
- **Corrección:** `require_role("admin")` en el servidor.

H6 – Exposición de datos en el perfil

- **Endpoint:** `GET /api/profile`
- **Condición:** se devolvía el registro completo.
- **Impacto:** salen `password_hash`, `role`, `internal_notes` y `payment_card`.
- **Evidencia:** `evidencias/BOLA_BFLA/antes.txt` → `despues.txt`
- **Corrección:** `ProfileOut` (username, full_name, email, address).

H7 – Asignación masiva en el perfil

- **Endpoint:** `PATCH /api/profile`
- **Condición:** `profile.update(payload)` con todo lo que llegaba en el body.
- **Impacto:** usuarioa se pone `role=admin` / `is_admin=true` y queda guardado.
- **Evidencia:** `evidencias/BOLA_BFLA/antes.txt` → `despues.txt` (422 y el perfil releído sin cambios)
- **Corrección:** `ProfileUpdate` con `extra="forbid"`.

H8 – JWT decodificado pero no validado

- **Endpoint:** todos los protegidos
- **Condición:** `jwt.decode(..., verify_signature=False)`.
- **Impacto:** se aceptan tokens con el payload alterado (rol admin inyectado), `alg=none`, de otro realm, ID tokens y tokens vencidos.
- **Evidencia:** `evidencias/JWT_OIDC/antes.txt` → `despues.txt`
- **Corrección:** RS256 contra la JWKS de Keycloak, `iss`, `aud`, `exp` y `typ=Bearer`, más scopes por endpoint. Prueba: `test_jwt.py`.

H9 – Pedidos sin validación de esquema

- **Endpoint:** `POST /api/orders`

- **Condición:** sin contrato. Se coercionan tipos y se aceptan campos extra.
- **Impacto:** se crean pedidos con `quantity` 0 u 11, con `"5"` como texto o sin dirección. Con `"price": 0` el pedido queda gratis.
- **Evidencia:** `evidencias/esquema_rate_limit/antes.txt` → `despues.txt`
- **Corrección:** `OrderCreate` (`strict`, `extra="forbid"`, rangos). Prueba: `test_schema.py`.

H10 – Sin límite de tasa

- **Endpoint:** `GET /api/orders/{id}`
- **Condición:** no había ningún límite.
- **Impacto:** enumeración ilimitada de IDs.
- **Evidencia:** `evidencias/esquema_rate_limit/antes.txt` → `despues.txt` (la petición 11 da 429)
- **Corrección:** ventana deslizante por `sub` (10/10 s) y `limit_req` en el gateway.

H11 – GraphQL sin límites

- **Endpoint:** `POST /graphql`
- **Condición:** sin límite de profundidad, de listas ni de costo, con la introspección abierta y sin control de dueño en los resolvers.
- **Impacto:** consultas de profundidad 6, `first: 50`, consultas de alto costo, introspección completa y el pedido de B visible para A.
- **Evidencia:** `evidencias/graphql/antes.txt` → `despues.txt`
- **Corrección:** `QueryDepthLimiter(3)`, regla de costo, `first ≤ 20`, introspección desactivada en prod y autorización en los resolvers. Prueba: `test_graphql.py`.

Caso Optus (sept. 2022) vs. mi laboratorio

Según la presentación, en Optus quedaron expuestos nombres, fechas de nacimiento, teléfonos, direcciones y, en algunos casos, documentos de identidad, a través de una API con protección insuficiente. No se intentó reproducir nada contra infraestructura real.

Qué se puede exponer si falla la autorización	Prueba de mi laboratorio que lo detecta	Control en el servidor	Evidencia de que funciona
Datos de otros clientes recorriendo IDs (nombre, dirección, pedidos)	BOLA: usuarioa pide `/api/orders/3`, que es de usuariob. En crAPI, ubicación de vehículos y pedidos ajenos	Comparar el dueño del objeto con el `sub` del token en cada endpoint. 404 igual para ajeno e inexistente	`test_bola_bfla.py::test_a_no_lee_pedido_de_b`, `test_ajeno_e_inexistente_responden_igual`. `BOLA_BFLA/antes.txt` da 200, `despues.txt` da 404
Todo lo anterior sin siquiera iniciar sesión	Llamar a la API sin token o con un token falso, alterado, `alg=none` o de otro emisor	Validar el JWT completo antes de ejecutar nada: firma RS256 con JWKS, `iss`, `aud`, `exp` y `typ`	`test_jwt.py` (sin token, alterado, vencido, otro realm, ID token: 401). `JWT_OIDC/antes.txt` vs `despues.txt`
Extracción masiva (millones de registros)	11 peticiones seguidas del mismo usuario; ráfaga de 300 contra el gateway	Rate limit por `sub` (10/10 s) en la API más `limit_req` por IP en el gateway	`test_rate_limit.py`: la petición 11 da 429 y B sigue con 200. `esquema_rate_limit/despues.txt`
Campos de más en la respuesta (hash, rol, notas internas, tarjeta, documento)	Comparar `GET /api/profile` con lo que de verdad necesita la pantalla. En crAPI, el feed de la comunidad devuelve el correo y el vehículo de otros usuarios	Modelo de respuesta con lista blanca (`ProfileOut`)	`test_perfil_no_expone_campos_internos`. `BOLA_BFLA/antes.txt` muestra la tarjeta y `despues.txt` no
Funciones internas o de administración alcanzables desde fuera	usuarioa llama a `/api/admin/audit`. Rutas como `/healthz` o `/docs` desde el gateway	Rol `admin` exigido en el servidor. El gateway solo publica una lista blanca de rutas	`test_usuario_comun_no_puede_ver_auditoria` (403), `test_rutas_no_publicadas` (404)
Un servicio interno expuesto que se salta el gateway	Conectar directo a `orders-api` sin certificado o con uno de otra CA. Revisar los puertos publicados	mTLS obligatorio (`CERT_REQUIRED`), red `backend` interna, sin `ports`	`test_mtls.py::test_api_rechaza_*`, `test_orders_api_no_publica_puertos`, `gateway_mTLS/host_segmentacion.txt`
Escalar privilegios modificando el propio perfil	`PATCH /api/profile` con `"rol e":"admin"` y releer el perfil	Lista blanca de campos editables (`extra="forbid"`)	`test_patch_perfil_con_rol e_admin_se_rechaza` (422, perfil sin cambios)

Resultado de pruebas automatizadas

Ejecución: `docker compose --profile test run --rm tests` — **75 passed**.

```
============================= test session starts ==============================
platform linux -- Python 3.12.16, pytest-8.3.4, pluggy-1.7.0 -- /usr/local/bin/python3.12
rootdir: /work/tests
plugins: anyio-4.16.0
collecting ... collected 75 items

test_bola_bfla.py::test_a_lee_su_pedido PASSED                           [  1%]
test_bola_bfla.py::test_a_no_lee_pedido_de_b PASSED                      [  2%]
test_bola_bfla.py::test_b_no_lee_pedido_de_a PASSED                      [  4%]
test_bola_bfla.py::test_ajeno_e_inexistente_responden_igual PASSED       [  5%]
test_bola_bfla.py::test_listado_solo_trae_lo_propio PASSED               [  6%]
test_bola_bfla.py::test_admin_puede_ver_auditoria PASSED                 [  8%]
test_bola_bfla.py::test_usuario_comun_no_puede_ver_auditoria PASSED      [  9%]
test_bola_bfla.py::test_auditoria_sin_token PASSED                       [ 10%]
test_bola_bfla.py::test_perfil_no_expone_campos_internos PASSED          [ 12%]
test_bola_bfla.py::test_patch_perfil_direccion PASSED                    [ 13%]
test_bola_bfla.py::test_patch_perfil_con_role_admin_se_rechaza PASSED    [ 14%]
test_bola_bfla.py::test_intentos_quedan_en_auditoria PASSED              [ 16%]
test_graphql.py::test_consulta_normal PASSED                             [ 17%]
test_graphql.py::test_paginacion PASSED                                  [ 18%]
test_graphql.py::test_a_no_ve_pedido_de_b_por_graphql PASSED             [ 20%]
test_graphql.py::test_profundidad_6 PASSED                               [ 21%]
test_graphql.py::test_first_50 PASSED                                    [ 22%]
test_graphql.py::test_first_500_por_variable PASSED                      [ 24%]
test_graphql.py::test_consulta_demasiado_costosa PASSED                  [ 25%]
test_graphql.py::test_introspeccion_apagada_en_prod PASSED               [ 26%]
test_graphql.py::test_graphql_sin_token PASSED                           [ 28%]
test_graphql.py::test_graphql_por_get_no PASSED                          [ 29%]
test_jwt.py::test_token_valido PASSED                                    [ 30%]
test_jwt.py::test_sin_token PASSED                                       [ 32%]
test_jwt.py::test_payload_alterado PASSED                                [ 33%]
test_jwt.py::test_payload_con_rol_admin_inyectado PASSED                 [ 34%]
test_jwt.py::test_firma_alterada PASSED                                  [ 36%]
test_jwt.py::test_alg_none PASSED                                        [ 37%]
test_jwt.py::test_hs256_firmado_con_la_clave_publica PASSED              [ 38%]
test_jwt.py::test_token_vencido PASSED                                   [ 40%]
test_jwt.py::test_token_de_otro_realm PASSED                             [ 41%]
test_jwt.py::test_id_token_no_sirve_como_access_token PASSED             [ 42%]
test_jwt.py::test_sin_scope_orders_read PASSED                           [ 44%]
test_jwt.py::test_scope_de_lectura_no_deja_crear PASSED                  [ 45%]
test_jwt.py::test_token_alterado_no_crea_pedido PASSED                   [ 46%]
test_jwt.py::test_cabecera_authorization_rara[Basic dXN1YXJpb2E6eA==] PASSED [ 48%]
test_jwt.py::test_cabecera_authorization_rara[Bearer] PASSED             [ 49%]
test_jwt.py::test_cabecera_authorization_rara[Bearer basura.basura.basura] PASSED [ 50%]
test_mtls.py::test_https_al_gateway PASSED                               [ 52%]
test_mtls.py::test_http_plano_al_gateway PASSED                          [ 53%]
test_mtls.py::test_cert_del_gateway_no_valida_con_otra_ca PASSED         [ 54%]
test_mtls.py::test_rutas_no_publicadas[/healthz] PASSED                  [ 56%]
test_mtls.py::test_rutas_no_publicadas[/docs] PASSED                     [ 57%]
test_mtls.py::test_rutas_no_publicadas[/openapi.json] PASSED             [ 58%]
test_mtls.py::test_rutas_no_publicadas[/api/internal] PASSED             [ 60%]
test_mtls.py::test_rutas_no_publicadas[/api/orders/../admin] PASSED      [ 61%]
test_mtls.py::test_rutas_no_publicadas[/api/orders/abc] PASSED           [ 62%]
test_mtls.py::test_rutas_no_publicadas[/] PASSED                         [ 64%]
test_mtls.py::test_metodo_no_permitido PASSED                            [ 65%]
test_mtls.py::test_bola_se_sigue_validando_en_el_servicio PASSED         [ 66%]
test_mtls.py::test_log_del_gateway_sin_token PASSED                      [ 68%]
test_mtls.py::test_api_acepta_cert_del_gateway PASSED                    [ 69%]
test_mtls.py::test_api_rechaza_sin_cert_de_cliente PASSED                [ 70%]
test_mtls.py::test_api_rechaza_cert_de_otra_ca PASSED                    [ 72%]
test_mtls.py::test_api_no_habla_http_plano PASSED                        [ 73%]
test_mtls.py::test_orders_api_no_publica_puertos PASSED                  [ 74%]
test_rate_limit.py::test_la_peticion_11_da_429_y_b_sigue_entrando PASSED [ 76%]
test_rate_limit.py::test_pasada_la_ventana_vuelve_a_entrar PASSED        [ 77%]
test_rate_limit.py::test_limit_req_del_gateway PASSED                    [ 78%]
test_schema.py::test_pedido_valido PASSED                                [ 80%]
test_schema.py::test_pedido_invalido[quantity_0] PASSED                  [ 81%]
test_schema.py::test_pedido_invalido[quantity_11] PASSED                 [ 82%]
test_schema.py::test_pedido_invalido[quantity_texto] PASSED              [ 84%]
test_schema.py::test_pedido_invalido[quantity_decimal] PASSED            [ 85%]
test_schema.py::test_pedido_invalido[quantity_bool] PASSED               [ 86%]
```

```
test_schema.py::test_pedido_invalido[sin_product_id] PASSED              [ 88%]
test_schema.py::test_pedido_invalido[product_id_en_blanco] PASSED        [ 89%]
test_schema.py::test_pedido_invalido[product_id_inexistente] PASSED      [ 90%]
test_schema.py::test_pedido_invalido[direccion_corta] PASSED             [ 92%]
test_schema.py::test_pedido_invalido[direccion_larga] PASSED             [ 93%]
test_schema.py::test_pedido_invalido[extra_price] PASSED                 [ 94%]
test_schema.py::test_pedido_invalido[extra_owner_id] PASSED              [ 96%]
test_schema.py::test_pedido_invalido[extra_status] PASSED                [ 97%]
test_schema.py::test_body_que_no_es_json PASSED                          [ 98%]
test_schema.py::test_body_gigante_lo_corta_el_gateway PASSED             [100%]

============================== 75 passed in 44.93s ==============================
```